



CSC 113 Project

-Phase2-

Report

Members:

- **Sham Ahmed Karkar (Leader) – 446207402**
Worked on: GUI
- **Cerine hanane Cheddad - 446207940**
Worked on: Linked Lists
- **Hadeel Mohammed - 445206292**
Worked on: Exceptions

Contents

Project idea:	3
Introduction:	3
Changes Made in Phase 2.....	3
Exception Handling.....	4
File Handling.....	4
GUI Design and Implementation:	5
GUI Components Used	5
GUI Design Strategy.....	6
The GUI frames:.....	7

Project idea:

A club management system.

Introduction:

This project is a club management system designed to organize and manage different aspects of a club, including committees, members, and events. The system allows users to add and manage committees, register different types of members such as board members and volunteers, and perform operations like adding or removing members, editing their information, and displaying detailed reports.

Changes Made in Phase 2

Several improvements and new features were added to the system during Phase 2:

- The data structure used in the project was modified. The arrays of Members, Committees, and Events were replaced with linked lists to improve flexibility and dynamic data management.
- Exception handling was added to make the program more reliable and able to handle invalid inputs and runtime errors properly.
- File handling was implemented using object serialization to allow saving and loading the system data from files.
- A graphical user interface (GUI) was developed to provide a more user-friendly interaction with the system instead of relying only on console-based interaction.
- Several classes and methods were updated and modified to support the new features and improve the overall functionality of the system.

Exception Handling

1. **DuplicateIdException (user-defined Exception)**

DuplicateIdException is a user-defined checked exception that is thrown when attempting to register a member or volunteer with an ID that already exists in the committee. The exception is handled using a try-catch block to display an appropriate error message and prevent duplicate registration.

2. **IllegalArgumentException (Unchecked Exception)**

IllegalArgumentException is an unchecked exception that is thrown when the user enters a negative number of volunteer hours. The exception is propagated to the main method and handled there using a try-catch block, where an error message is displayed and the user is asked to enter a valid value.

3. **IOException / ClassNotFoundException (Checked Exceptions)**

IOException and ClassNotFoundException are built-in checked exceptions used while loading the club data from the file. They are handled using try-catch block in the methods where they occurred.

File Handling

The program uses an **object file** named clubData.dat to permanently store the club information. File handling is implemented using Java object serialization, which allows the entire Club object and its associated data to be saved and restored.

- The **saveData(Club club)** method is responsible for saving the data. It uses FileOutputStream together with ObjectOutputStream to write the Club object into the file using the writeObject() method.
- The **loadData()** method is responsible for retrieving previously saved data. It uses FileInputStream together with ObjectInputStream to read the serialized Club object from the file using the readObject() method.
- Exception handling is included to ensure the program behaves correctly during file operations, such as handling missing files and errors during reading or writing.

GUI Design and Implementation:

The graphical user interface (GUI) of the system was implemented using the Java Swing library. The design focused on consistency, usability, and modularity to create a professional and organized application interface. The GUI consists of multiple frames, including:

- a main menu frame for navigating through the system,
- a frame for adding committees,
- a frame for adding new members,
- a frame for editing member information,
- a frame for managing events,
- and a reports dashboard frame for displaying reports.

User interactions are handled using event-driven programming through ActionListener events attached to buttons and selection components.

GUI Components Used

Several Swing components were used throughout the system:

- **JFrame:**
Used as the main top-level container for all application windows such as the main menu, reports page, event management, and member forms.
- **JPanel:**
Panels were used to organize the interface into sections. The application mainly uses a two-panel structure consisting of a dark-green navigation area and a light-gray workspace area.

- **JButton:**
Buttons were implemented to handle user actions such as adding members, saving data, generating reports, managing events, and exiting the system.
- **TextField:**
Used for user input including member IDs, names, committee names, dates, and locations.
- **JTextArea and JScrollPane:**
The reports section uses a JTextArea inside a JScrollPane to allow users to display and scroll through long reports easily.
- **JRadioButton and ButtonGroup:**
Used to allow users to select the member type (Volunteer or Board Member) while ensuring only one option can be selected at a time.
- **JComboBox:**
Used for selecting predefined options such as report types and board member positions.
- **JCheckBox:**
Used to represent boolean values such as the active status of members.
- **JOptionPane:**
Implemented for displaying success messages, warnings, and error dialogs to improve user interaction and error handling.

GUI Design Strategy

The GUI design was based on consistency and code reusability.

To achieve a uniform design across all windows, a parent class named BaseFrame was created. This class contains all shared design properties such as:

- frame size,
- panel colors,
- common buttons,

- layout structure,
- and reusable methods.

All other forms inherit from BaseFrame, which reduced code duplication and maintained a consistent appearance throughout the application.

A reusable method called addField() was also implemented to simplify the process of adding labels and input components to forms. This improved readability and reduced repetitive code.

The interface uses a dark-green color theme (Color(0,45,45)) combined with light-gray workspaces to create a clean and professional visual appearance with high contrast and readability.

The application uses a **Null Layout** (setLayout(null)), which allows precise manual positioning of components using the setBounds() method. This approach provided full control over the placement and alignment of GUI elements according to the required design.

The GUI frames



Main Menu

1. Add Committee

2. Add New Member

3. Edit Member Info

4. View Reports

5. Manage Events

6. Save & Exit

 Add Committee



Add Committee

Save

Name:

 Add Member Form



Add Member Form

ADD Member

Committee Name:

ID:

Name:

Year:

2026

Active?

☐ Active Member

Type

☐ Volunteer ☐ Board

Edit Member Info

Save Changes

Committee Name:

Member ID:

New Name:

Status:

Find Member

☐ Active Status

System Reports

Run Report

Active Members: 0

Type:

Member Report

Committee:

ID/Event:

Manage Events

Save Event

Event Name:

Date:

Location: